ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Formal Methods for Information Security

Project Report

Mehdi Ziazi , Adam Mernissi Arifi

May 31, 2024

Professors : Dr. Ralf Sasse, Dr. Christoph Sprenger and Dr. Srdan Krstic

Department of Computer Science, ETH Zürich

Contents

Contents **i**

1 PACE Protocol **1**
1.1 A simple challenge-response protocol 1
1.2 Mutual authentication 1
1.3 Introducing a session key 4
1.4 Replace the password by a nonce 8
1.5 Introducing Diffie-Hellman: The PACE protocol 9

2 The Off-the-Record Messaging Protocol **14**
2.1 Modeling the original OTR Key Exchange 14
2.2 Authentication Failure 15
2.3 Improvement 19
2.4 SIGMA 20

Chapter 1

PACE Protocol

1.1 A simple challenge-response protocol

We try to model the following protocol in our theory P1:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} \quad & : x \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : [x]_{k(\mathrm{B},\mathrm{A})} \end{aligned}$$

Discussion :

Since the key generation rule **Share_Unidirectional_Key** generates a unidirectional shared key between two participants using the **Fr** rule, and since our participants initiation rules (**Init_A** and **Init_B**) use two of these keys (One to send from A to B and the other to send from B to A), we are guaranteed by the **Fr** rule that the keys k(A,B) and k(B,A) are fresh and therefore $k(A,B) \neq k(B,A)$.
Moreover, the addition of the **Reveal** rule enables the intruder to act as a participant in the protocol.
Finally , we can see from the proof that the authentication property that A injectively agrees with B on the nonce x is verified for the protocol.

1.2 Mutual authentication

We try to model the following protocol in our theory P2:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} \quad & : x \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : y \\ \mathrm{A} \rightarrow \mathrm{B} \quad & : [y]_{k(\mathrm{A},\mathrm{B})} \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : [x]_{k(\mathrm{B},\mathrm{A})} \end{aligned}$$

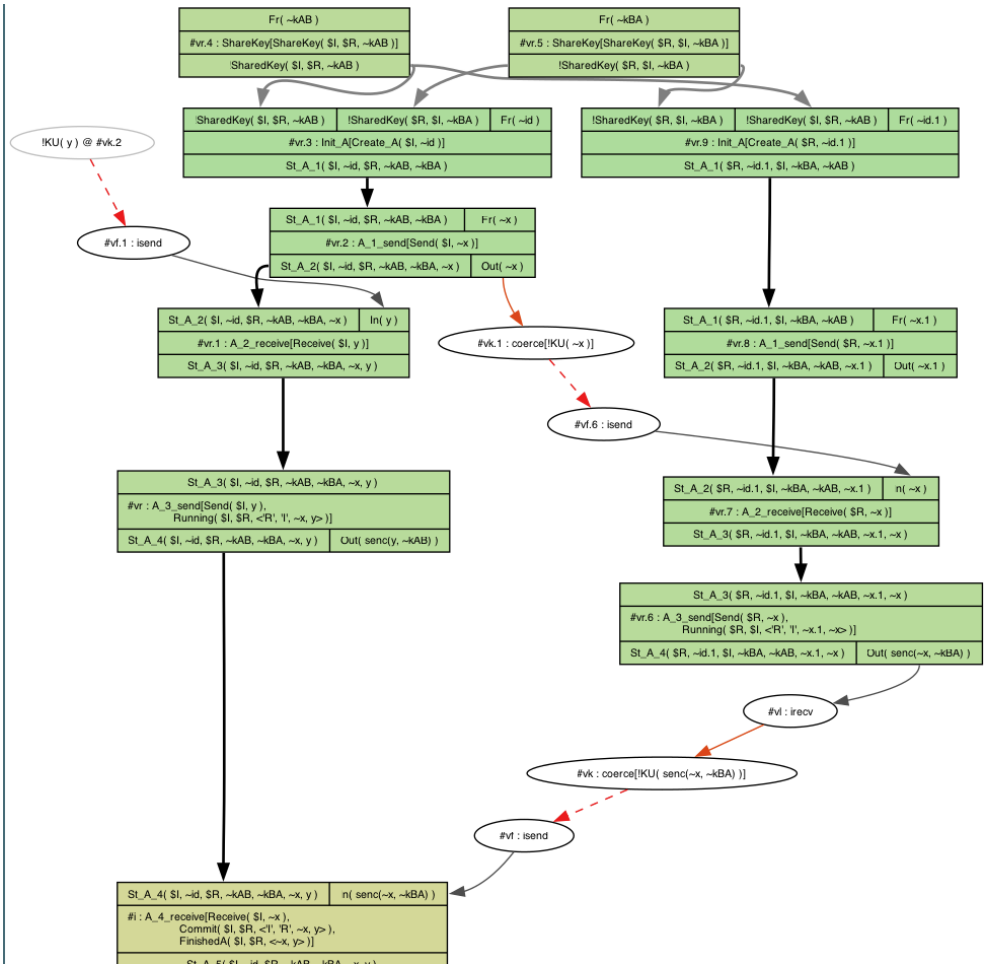


Figure 1.1: Attack on the (non-)injective agreement of the Initiator

(a) We can see that neither the (non-)injective agreement of the initiator nor responder holds.

We will elaborate on this attack against the Initiator:
Let A1, A2 and I denote the communication parties and the Intruder respectively in the protocol .

- As an Initiator, A1 wants to talk to A2, who will be the Responder.
- Similarly, as an Initiator, A2 wants to talk to A1, who will be the Responder.
- The Attacker will trick A1, as an Initiator, to think that he agreed with A2 on some arbitrary nonces, when such agreement never occured .

More precisely, the protocol begins with A1 sending its nonce x to A2 :

$$\text{A1} \rightarrow \text{A2} : x$$

Similarly, A2 will initiate a run of the protocol with A1 by sending its nonce y, but the Intruder intercepts it and drops it:

$$\text{A2} \rightarrow \text{I} : Drop(y)$$

The intruder replies to A1 with another nonce of its choice w:

$$\text{I} \rightarrow \text{A1} : w$$

Then A1 sends the encrypted nonce w to A2, but the intruder intercepts it and drops it:

$$\text{A1} \rightarrow \text{I} : Drop([w]_{k(\text{A1,A2})})$$

Finally, A2 sends the encrypted nonce x back to A1 :

$$\text{A2} \rightarrow \text{A1} : [x]_{k(\text{A2,A1})}$$

A1 then completes the protocol (and commits the nonce w) thinking it was running as the Initiator with A2 as the Receiver, and that they agreed on the nonces x and w, while A2 was also an Initiator and has not sent w. The attack on the (non-)injective agreement of the Receiver is the same except that the Intruder manipulates the nonce x this time with A2 running as the Receiver. Note that we added the following restrictions in P2a to obtain the minimal described attack:

restriction onedirectionnalsharedkey:
All $A\,B\,k1\,k2\,\#i\,\#j$. $\text{ShareKey}(A,B,k1)@i \wedge \text{ShareKey}(A,B,k2)@j \implies \#i = \#j \wedge k1 = k2$

restriction only1threadsA:
All $A\,\text{id1}\,\text{id2}\,\#i\,\#j$. $\text{Create_A}(A,\text{id1})@i \wedge \text{Create_A}(A,\text{id2})@j \implies \#i = \#j \wedge \text{id1} = \text{id2}$

restriction onlyonerole:
All $X\,\text{id1}\,\text{id2}\,\#i\,\#j$. $\text{Create_A}(X,\text{id1})@i \wedge \text{Create_B}(X,\text{id2})@j \implies F$

restriction only1threadsB:
All $B\,\text{id1}\,\text{id2}\,\#i\,\#j$. $\text{Create_B}(B,\text{id1})@i \wedge \text{Create_B}(B,\text{id2})@j \implies \#i = \#j \wedge \text{id1} = \text{id2}$

(b) We define our fix theory P2b as follow:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} \quad & : x \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : y \\ \mathrm{A} \rightarrow \mathrm{B} \quad & : [\text{'I'}, y, x]_{k(A,B)} \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : [\text{'R'}, x, y]_{k(B,A)} \end{aligned}$$

In the previous attack, we have seen that the Intruder breaks the (non-)injective agreement of P2 , this is due to two reasons :

- Possibility of injecting a new nonce (w), and sending it to one of the communication parties (A1), while having the other party (A2) running with its own generated nonce (y) .
- Possibility of making two parties of the same role (Initiator or Responder) communicate with each other .

Therefore, as a fix, each participant expects, in addition to its own encrypted nonce, the nonce of the other party (encrypted to avoid tampering from the intruder) to be sent back, to avoid committing over a different nonce than the one generated by the other party . Moreover, each participant also expects the role played by the other party in the protocol (to avoid having two same roles completing a run of the protocol) . With this fix, we can see that the mutual injective agreement property is verified.

1.3 Introducing a session key

We define our refinement theory P3a as follows:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} \quad & : x \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : y \\ \mathrm{A} \rightarrow \mathrm{B} \quad & : [\text{'I'}, \mathrm{A}, y]_{\mathrm{Kab}} \\ \mathrm{B} \rightarrow \mathrm{A} \quad & : [\text{'R'}, \mathrm{B}, x]_{\mathrm{Kab}} \end{aligned}$$

(a) This protocol is defined similarly to P2a, except that now, the headers 'I' for Initiator and 'R' for Responder are added in the third message. This is done to prevent the attack discussed in 1.2 b), which makes two parties of the same role (Initiator or Receiver) communicate with each other and complete a run of the protocol.
The second refinement is to add the actual identity of the sender/receiver in the encrypted message, this is done to prevent the attack

(Figure 1.2) described below :

Let's consider the following configuration:

- X_I (X in the role A/Initiator) wants to communicate with Y_R (Y in the role B/Receiver)
- Y_I (same identity Y, in the role A/Initiator) wants to communicate X_R (same identity X, in the role B/Receiver).

The attack begins with the Intruder I intercepting the nonce x of X_I and sending it to X_R:

$$\begin{aligned} X_I \rightarrow \mathrm{I} &: x \\ \mathrm{I} \rightarrow X_R &: x \end{aligned}$$

X_R, thinking Y_I initiated a run of the protocol, replies to Y_I with its nonce x_R, the Intruder intercepts it once again, and sends it to X_I (instead of Y_I) :

$$\begin{aligned} X_R \rightarrow \mathrm{I} &: y \\ \mathrm{I} \rightarrow X_I &: y \end{aligned}$$

Then, X_I sends "back" to Y_R the message ['I',y], where 'I' is the header for the Initiator and y is the nonce of X_R . This message is encrypted with Kxy = kdf(k(X,Y),x,y), where k(X,Y) is a shared symmetric key between X and Y (no longer an uni-directional key like in P2), x and y are the nonces of X_I and X_R (not Y_R) respectively, the Intruder intercepts the message and sends it to X_R:

$$\begin{aligned} X_I \rightarrow \mathrm{I} &: [\text{'I'}, y]_{\mathrm{Kxy}} \\ \mathrm{I} \rightarrow X_R &: [\text{'I'}, y]_{\mathrm{Kxy}} \end{aligned}$$

Since k(X,Y) is a shared bi-directional symmetric key between X and Y, the derived key Kxy is also shared bi-directional (in contrary to P2 where the messages were encrypted with a uni-directional key), this tricks X_R to accept it as a message coming from Y_I and send the correct reply, the Intruder redirects it to X_I:

$$\begin{aligned} X_R \rightarrow \mathrm{I} &: [\text{'R'}, x]_{\mathrm{Kxy}} \\ \mathrm{I} \rightarrow X_I &: [\text{'R'}, x]_{\mathrm{Kxy}} \end{aligned}$$

Again, because of the construction of Kxy (which now is bi-directional and no longer uni-directional), X_I completes the run of the protocol thinking he was talking with Y_R where he was talking with himself .

To summarize , we can see that the main issue is the replacement of the 2 uni-directional keys with one bi-directional symmetric key Kxy. This enables the Intruder to trick X (in both the Initiator and the Responder roles) to think he completed the protocol with another party Y where the latter was never alive during the protocol to begin with! Therefore falsifying the (non-)injective agreement property.

To address this issue in the theory P3a , both the Initiator and the Receiver need to send their identity in the encrypted messages (e.g for the Initiator : $[' I', A, y]_{K_{xy}}$). With this fix, we are able to prove that all the desired lemmas hold.

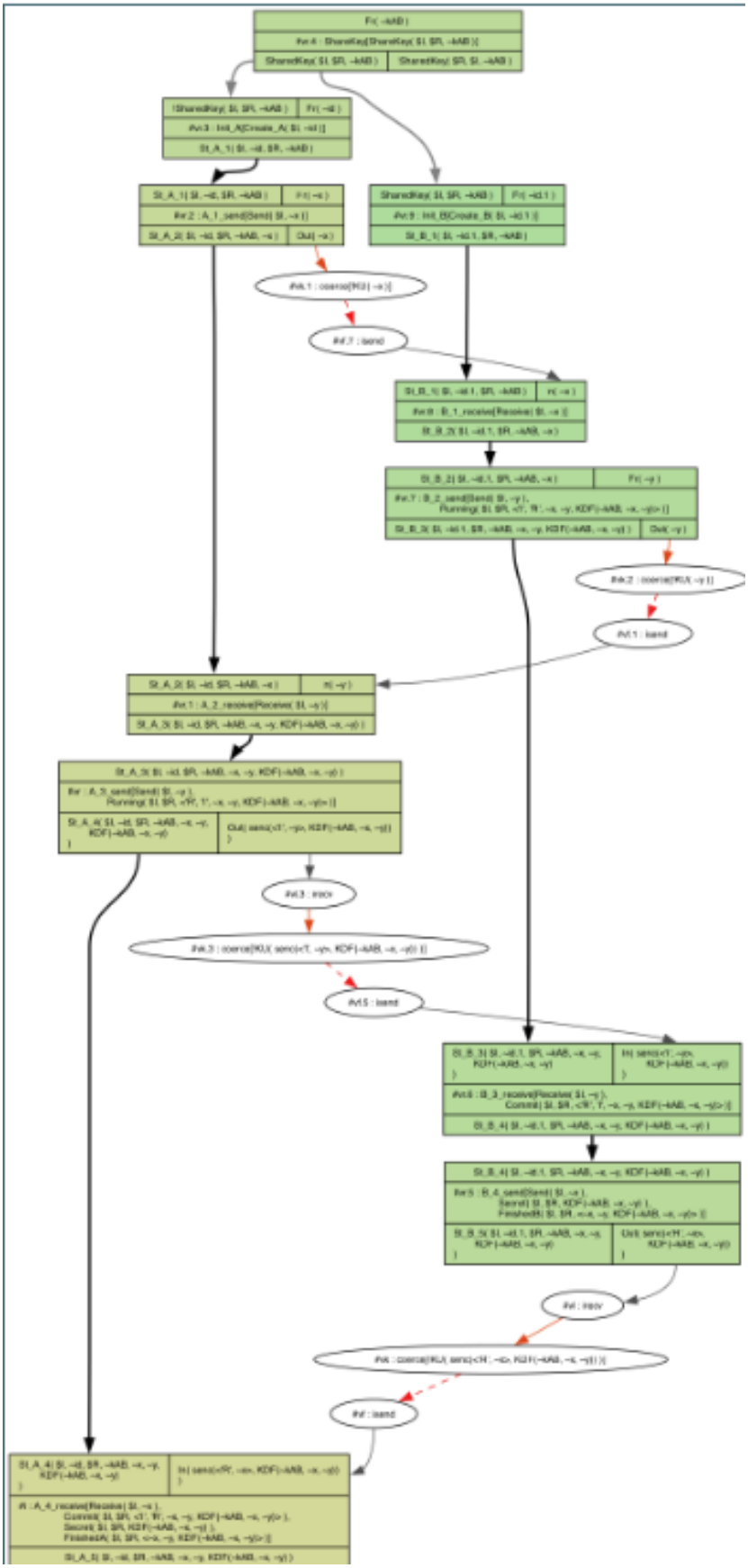


Figure 1.2: Attack against (non-)injective agreement of the initiator and receiver when identity isn't clearly sent in P3a

(b) The reason why we include in P3a each role's own nonce in the agreement despite that nonce not being maced derives from the construction of the protocol key Kab. The key is generated using both nonces, which means that if A and B have seen different nonces, they won't derive the same key Kab and therefore won't be able to decrypt the messages received and commit on the messages exchanged. However, in the protocol P2, the keys used in the protocol were the pre-shared unidirectional keys which would result in a correct decryption regardless of the nonces exchanged , and therefore the parties would commit on corrupted nonces if they weren't maced .

1.4 Replace the password by a nonce

We model the theory P4 as follows:

$$\begin{aligned} \mathrm{A} \to \mathrm{B} &: \langle x, [s]_{h(k(\mathrm{A},\mathrm{B})} \rangle \\ \mathrm{B} \to \mathrm{A} &: y \\ \mathrm{A} \to \mathrm{B} &: [\text{'I'}, \mathrm{A}, y]_{\mathrm{Kab}} \\ \mathrm{B} \to \mathrm{A} &: [\text{'R'}, \mathrm{B}, x]_{\mathrm{Kab}} \end{aligned}$$

Discussion :

We can see that after running the theory, all the authentication lemmas as well as secrecy hold for P4, just like P3. We can (informally) argue by pointing out that the two protocols P3a and P4 are similar, and showing that the differences between the two them don't enable the intruder to perform any attack against the desired properties in P4.

For the similarities, we can see that both protocols rely on a kdf to derive a session key. They also both include the identities of the participants, same headers 'I' and 'R', as well as two nonces x and y sent in the clear. For the differences, we see that the kdf from protocol P3a uses the pre-shared key k(A,B), while the kdf in P4 uses a freshly generated nonce instead. However, the newly introduced nonce "s" is secret since it is sent encrypted with the hash of k(A,B), making sure that only A and B can know the nonce and derive Kab and therefore making "s" play the same role as k(A,B) in P3a (except s has a higher entropy than k(A,B) and is therefore better to use).

1.5 Introducing Diffie-Hellman: The PACE protocol

We transform the model P4 into the following model in the theory P5ab:

$$\begin{aligned} \text{A} \to \text{B} \quad &: \langle g^x, [s]_{h(k(\text{A,B}))} \rangle \\ \text{B} \to \text{A} \quad &: g^y \\ \text{A} \to \text{B} \quad &: [\text{'I'}, \text{A}, g^y]_{K_{\text{ab}}} \\ \text{B} \to \text{A} \quad &: [\text{'R'}, \text{B}, g^x]_{K_{\text{ab}}} \end{aligned}$$

(a) The protocol P5ab replaces the nonces x and y by Diffie-Hellman half-keys g^x and g^y where ″g″, ″x″ and ″y″ are the base/generator, the private key of A and the private key of B, respectively. The secret nonce ″s″ is no longer used in a kdf for the key derivation, but used instead for the derivation of the base ″g″ , along with a public domain parameter ″p″ , which gives g = map(s,p), where map is a user-defined hash function without equational properties. Moreover, the session key Kab depends now only on the secret key g^{xy}, established from the exchange of the Diffie-Hellman half-keys . We also don't require A to send ″p″ to B in the first message, since it is publicly available and modeled as such in Tamarin ($p) . Finally, we added the following restriction in our theory P5ab:

restriction notNeutral:

$$\text{All } x \ \#i.\ \text{NotNeutral}(x)@i \implies \neg(x = \text{DH_neutral})$$

The purpose of this restriction is to remove the trivial Man-in-the-middle attack (MITM), rising from the attacker replacing g^x and g^y by the neutral element, which leads to both parties deriving the DH neutral element as symmetric key (which is no longer a secret) . Hence, we make each of A and B check that the public keys received are valid using this restriction. Finally, note that this is equivalent to pattern matching on both public key shares in the rules containing an Input fact.

(b) After adding the Perfect Forward Secrecy lemma to our protocol P5ab, we can see that the property holds without any modification to the protocol. This can be argued by the use of Ephemeral Diffie-Hellman, where in each run of the protocol, we generate new private/public keys (x,g^x) without using the long-term symmetric key k(A,B).
Moreover, the compromise of k(A,B) will only leak ″s″ (and therefore

the base "g"), so the attacker would still need to solve the Discrete Logarithm Problem, which we assume he cannot. Hence, the attacker cannot break the Perfect Forward Secrecy of the protocol.

(c) The secrecy of the base g is essential in the protocol. To prove this, we keep the same protocol P5ab with the addition of outputting the nonce "s" in the first message, so that g becomes public. Suppose the following configuration where A wants to run the protocol with B, and I is the intruder .

A sends to B its Diffie-Hellman share g^x, the nonce s (in clear) and the encrypted nonce with the hash of the pre-shared symmetric key, the intruder I intercepts the message:

$$\mathrm{A} \rightarrow \mathrm{I} : \langle g^x, s, [s]_{h(K(\mathrm{A},\mathrm{B}))} \rangle$$

The intruder generates a fresh Diffie-Hellman key-pair (w , g^w) and sends back its public key to A:

$$\mathrm{I} \rightarrow \mathrm{A} : g^w$$

A then sends to B the compromised public Diffie-Hellman key, the Intruder intercepts it:

$$\mathrm{A} \rightarrow \mathrm{I} : [\text{'I'}, \mathrm{A}, g^w]_{K_{\mathrm{ab}}}$$

Now, since the Intruder established the Diffie-Hellman shared key ($g^x w$) with A , he knows it and therefore can compute the session key Kab= h($g^{(}xw)$) and send the final message with the identity of B:

$$\mathrm{I} \rightarrow \mathrm{A} : [\text{'R'}, \mathrm{B}, g^x]_{K_{\mathrm{ab}}}$$

A then completes the protocol, but none of (perfect forward)secrecy nor (non-)injective agreement holds. Actually, B was never alive to begin with! The same mirror attack is possible against B .

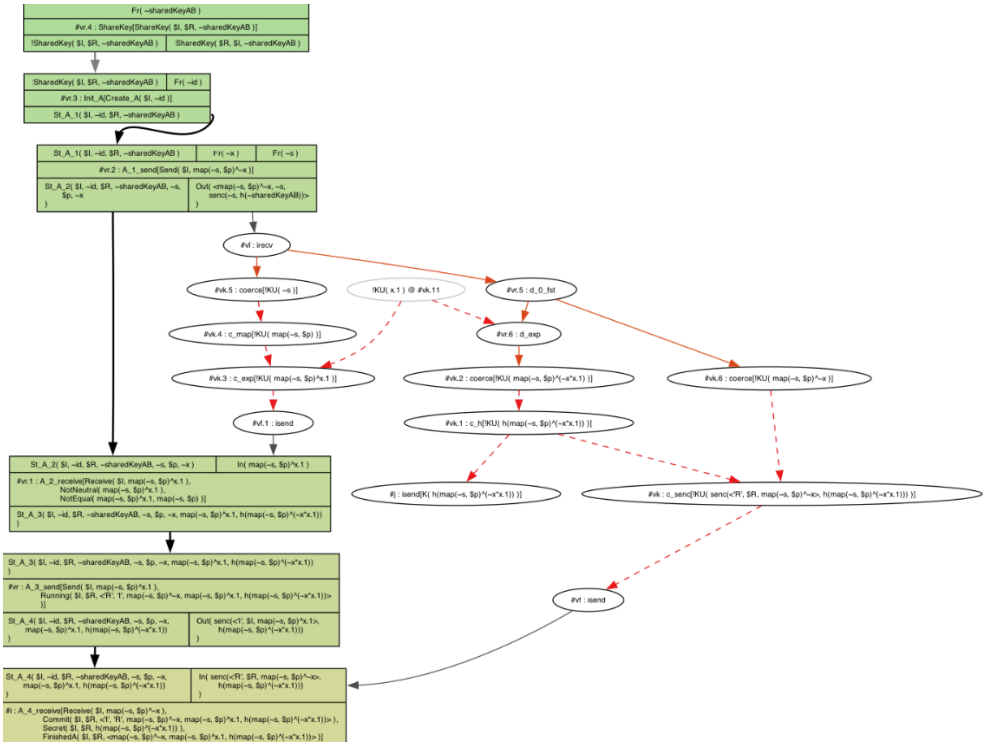


Figure 1.3: Attack described in 1.5.c if the base "g" is not secret

(d) When we remove all the tags, we consider the following protocol with the last two messages are unifiable:

$$\begin{aligned} \mathrm{A} \to \mathrm{B} \quad &: \langle g^x, [s]_{h(K(\mathrm{A},\mathrm{B}))} \rangle \\ \mathrm{B} \to \mathrm{A} \quad &: g^y \\ \mathrm{A} \to \mathrm{B} \quad &: [\langle A, g^y \rangle]_{K_{\mathrm{ab}}} \\ \mathrm{B} \to \mathrm{A} \quad &: [\langle B, g^x \rangle]_{K_{\mathrm{ab}}} \end{aligned}$$

Discussion :

Since we consider that agents may talk to themselves, let's consider an attack in the following setting (Note that the indices I/R are only here to differentiate between each of the thread where X is an Initiator/Responder, but it's still the same agent with a fixed identity):

X_I wants to communicate with X_R using the shared key of X with himself: k(X,X). Hence, he initiates the run of the protocol by generating a fresh DH key-pair and a fresh nonce "s" for computing the base, he then sends the DH public key, and "s" encrypted with the hash(k(x,x))

the Intruder I intercepts it and sends him back its own DH public key share:

$$\begin{aligned} X_I \rightarrow \mathrm{I} \quad &: \langle g^x, [s]_{h(k(\mathrm{X},\mathrm{X}))} \rangle \\ \mathrm{I} \rightarrow X_I \quad &: g^x \end{aligned}$$

Then, X_I sends back the challenge (its identity X and the received DH share, encrypted with the k(X,X)) and expects the receiver to send its own challenge, but since the last two messages are unifiable, and X is communicating with itself, the sent challenge is the same as the expected one (same identity and key share). The intruder can then just intercept it and send it back:

$$\begin{aligned} X_I \rightarrow \mathrm{I} \quad &: [\langle X, g^x \rangle]_{K_{\mathrm{xx}}} \\ \mathrm{I} \rightarrow X_I \quad &: [\langle X, g^x \rangle]_{K_{\mathrm{xx}}} \end{aligned}$$

Consequently, the initiator X_I completes a run of the protocol without the participation of the receiver X_R (X never ran as Receiver during the protocol) breaking the (non-)injective agreement of the Initiator. Finally, this attack is only possible on the Initiator since we need to intercept the DH public key and send it back, which cannot be done for the Receiver because he expects this.

After adding the following restriction to our protocol P5d:

restriction notequalhalfsharedkeys:

$$\text{All } x\, y\, \#i.\ \mathrm{NotEq}(x, y)@i \implies \neg(x = y)$$

This attack is not possible anymore and all the desired properties hold.

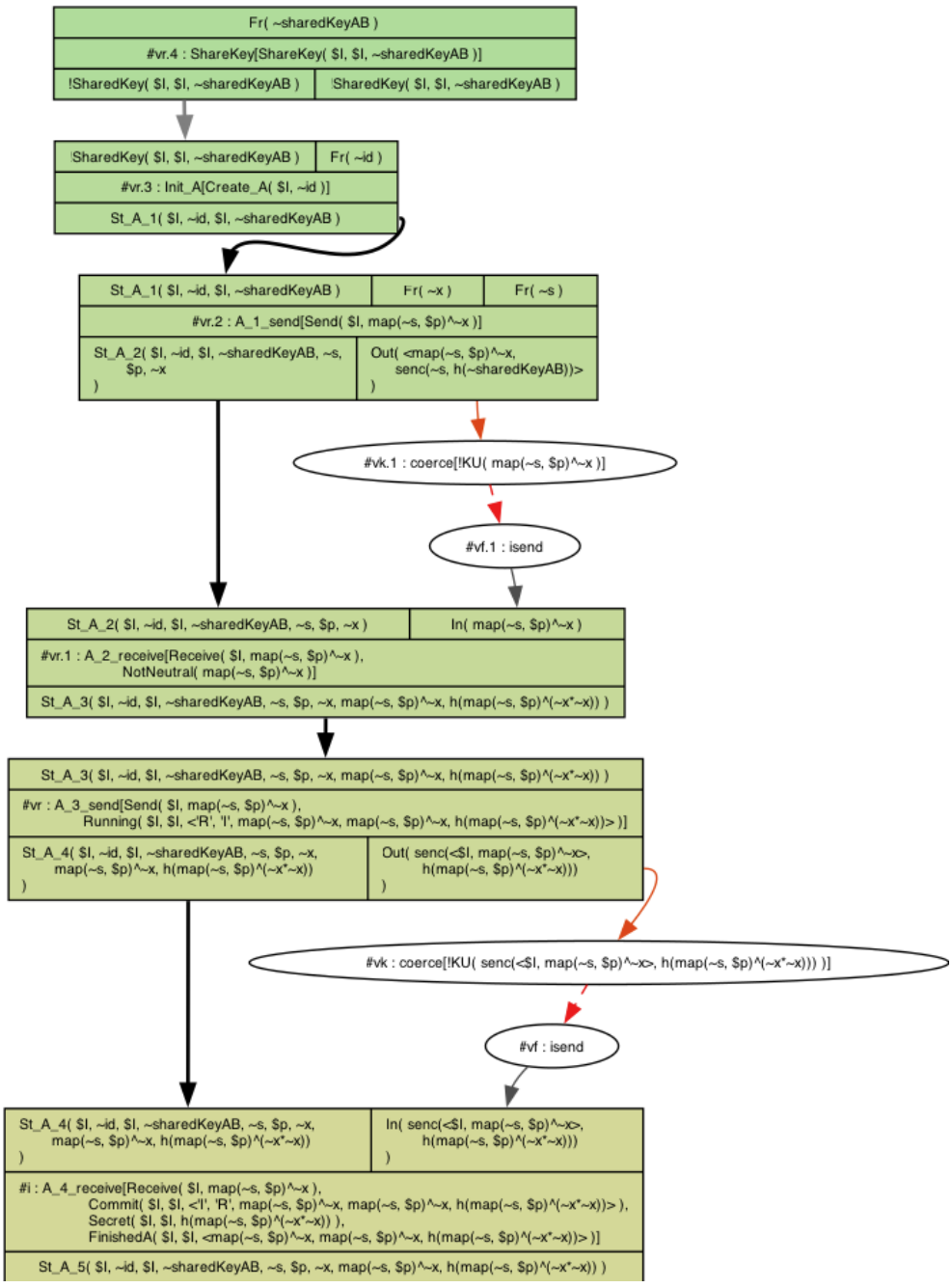


Figure 1.4: Attack against the (non-)injective agreement of the Initiator in P5d before the fix

Chapter 2

The Off-the-Record Messaging Protocol

2.1 Modeling the original OTR Key Exchange

The OTR protocol is defined in the paper as:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} &: \mathrm{Sign}(g^x, \mathrm{sk}_A), \mathrm{vk}_A \\ \mathrm{A} \leftarrow \mathrm{B} &: \mathrm{Sign}(g^y, \mathrm{sk}_B), \mathrm{vk}_B \end{aligned}$$

We try to model it in the theory OTR1 as follows:

$$\begin{aligned} \mathrm{A} \rightarrow \mathrm{B} &: \langle g^x, \mathrm{sign}(g^x, \mathrm{sk}_A), pk(\mathrm{sk}_A)) \rangle \\ \mathrm{B} \rightarrow \mathrm{A} &: \langle g^y, \mathrm{sign}(g^y, \mathrm{sk}_B), pk(\mathrm{sk}_B) \rangle \end{aligned}$$

Discussion :

In the OTR Authenticated Key-Exchange part of the paper, they assume that the receiver is able to recover the Diffie-Hellman share from the signature, requiring only the publicly available key of the sender to verify the validity of the signature, which implies that:

$$\mathrm{Sign}(g^w, \mathrm{sk}_X), \mathrm{vk}_X \equiv \langle g^w, \mathrm{sign}(g^w, \mathrm{sk}_X), pk(\mathrm{sk}_X)) \rangle$$

We could argue that they are not necessarily equivalent and could be invalid in a real world implementation (e.g DSA), since it is not always guaranteed to extract a message from a signature but may need to send the message too to be able to verify it . By adding the message, both the sender and receiver can

verify the validity of the signature, using the public key which is associated to the identity of the specific party. Finally, both parties can compute their shared secret value g^{xy}, this justifies the correctness of the protocol (with the addition of the message) .

2.2 Authentication Failure

The MITM attack described in the paper is:

$$\begin{array}{lcll} A & \rightarrow & E[B] & : g^x, \text{Sign}_{sk_A}(g^x), vk_A \\ E & \rightarrow & B & : g^x, \text{Sign}_{sk_E}(g^x), vk_E \\ E & \leftarrow & B & : g^y, \text{Sign}_{sk_B}(g^y), vk_B \\ A & \leftarrow & E[B] & : g^y, \text{Sign}_{sk_B}(g^y), vk_B \end{array}$$

We are able to reproduce this attack on Tamarin using this authentication lemma:

lemma keyIsAssociatedWithRightIdentity:

$$\begin{aligned} &\text{All } A\ E\ B\ k\ \#i\ \#j.\ \text{KeyIdentityAssociation}(A, B, k)@i \wedge \text{KeyIdentityAssociation}(B, E, k)@j \\ &\wedge \neg\left((\exists r.\text{Reveal}(A)@r) \vee (\exists r.\text{Reveal}(B)@r)\right) \\ &\implies A = E \end{aligned}$$

This lemma captures the fact that, if a key is established by two Agents A and B, then it is indeed associated with their identity and is "MITM free". More specifically, if A derived a key with B from the DH exchange, and B derived the same exact key, then B must have done so with A and not with another agent. Therefore, it becomes impossible to forward signed messages to sessions in which different identities are involved. (We, of course, exclude the traces where A or B got compromised).

In our protocol, we coupled our lemma with some restrictions to obtain a nice attack trace (the figure below):

restriction notalkwithself:
All $A\,B\,k\,\#i$. KeyIdentityAssociation$(A, B, k)@i \implies \neg(A = B)$

restriction onekeypername:
All $A\,\#i\,\#j$. Register$(A)@i \wedge$ Register$(A)@j \implies \#i = \#j$

restriction oneInitiator:
All $A\,B$ id1 id2 $\#i\,\#j$. Create_A$(A,$ id1$)@i \wedge$ Create_A$(B,$ id2$)@j$
$\implies A = B \wedge \#i = \#j \wedge$ id1 $=$ id2

restriction oneResponder:
All $A\,B$ id1 id2 $\#i\,\#j$. Create_B$(A,$ id1$)@i \wedge$ Create_B$(B,$ id2$)@j$
$\implies A = B \wedge \#i = \#j \wedge$ id1 $=$ id2

The attack goes as follow:

A sends to B its DH public key (g^x), the signature and corresponding public key to verify it, the intruder intercepts it:

$$\text{A} \rightarrow \text{I} : \langle g^x, \text{sign}(g^x, \text{sk}_A), pk(\text{sk}_A)) \rangle$$

In the paper, they assumed that the intruder has a valid private/public key for signing. In Tamarin, this is modeled by the fact that the Intruder uses the Reveal rule on another external agent (different from A and B) to get its private key (and public of course). Let **M**, **sk_M** and **pk(sk_M)** be the external revealed agent,its private signing key and its corresponding public key, respectively . The Intruder relays the DH share to B and signs using the newly revealed key:

$$\text{I} \rightarrow \text{B} : \langle g^x, \text{sign}(g^x, \text{sk}_M), pk(\text{sk}_M)) \rangle$$

Then B replies to the Intruder (to M in Tamarin and it gets intercepted by the Intruder) with its DH key share:

$$\text{B} \rightarrow \text{I} : \langle g^y, \text{sign}(g^y, \text{sk}_B), pk(\text{sk}_B)) \rangle$$

The Intruder then forwards it to A:

$$\mathrm{I} \rightarrow \mathrm{A} : \langle g^y, \mathrm{sign}(g^y, \mathrm{sk}_B), pk(\mathrm{sk}_B)) \rangle$$

Finally, A completes the protocol claiming B established the key with him, while B established the key with the Intruder (the revealed M in Tamarin), which falsifies our keyIsAssociatedWithRightIdentity lemma. Moreover, since B establishes a key with the Intruder (with M that got revealed in Tamarin), we can't expect anything from the keyIsAssociatedWithRightIdentity lemma and is therefore verified for B.

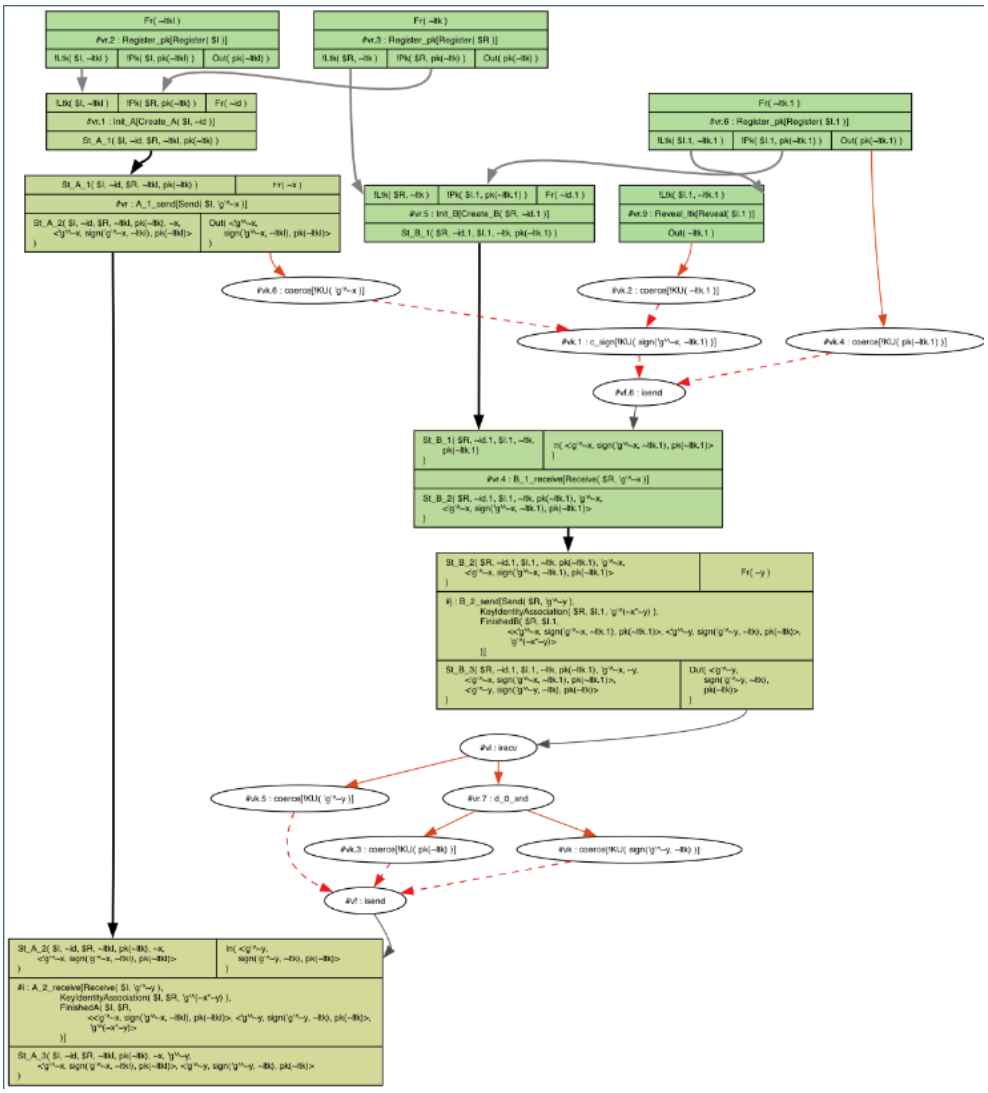


Figure 2.1: MITM attack against OTR

2.3 Improvement

We model the improved protocol in our theory OTR3:

$$\begin{aligned} \text{A} \rightarrow \text{B} &: g^x, \text{sign}(\langle g^x, B\rangle, \text{sk}_A), pk(\text{sk}_A) \\ \text{B} \rightarrow \text{A} &: g^y, \text{sign}(\langle g^y, A\rangle, \text{sk}_B), pk(\text{sk}_B) \end{aligned}$$

Discussion :

As described in the paper, including the identity of the intended Receiver in the signature removes the MITM attack described previously .
However, none of the standard authentication lemmas hold for any role. We will present an attack against the (non-)injective agreement of the Initiator, the one against the Receiver is similar. Let's consider the following configuration:

- X_I (X in the role A/Initiator) wants to communicate with Y_R (Y in the role B/Receiver)
- Y_I (same identity Y, in the role A/Initiator) wants to communicate X_R (same identity X, in the role B/Receiver).

X_I begins by sending its share to Y_R with the signature, the Intruder intercepts it and drops it. Y_I sends its share to X_R too which the Intruder intercepts and forwards to X_I:

$$\begin{aligned} X_I \rightarrow \text{I} &: Drop(g^x, \text{sign}(\langle g^x, X\rangle, \text{sk}_X), pk(\text{sk}_X) \\ Y_I \rightarrow \text{I} &: g^y, \text{sign}(\langle g^y, Y\rangle, \text{sk}_Y), pk(\text{sk}_Y) \\ \text{I} \rightarrow X_I &: g^y, \text{sign}(\langle g^y, Y\rangle, \text{sk}_Y), pk(\text{sk}_Y) \end{aligned}$$

X_I (X in the role Initiator) completes the run of the protocol with Y_I (Y in the role initiator), and not with the intended Y_R (Y in role receiver) but. Moreover, no agent in the role Responder ran the protocol, thus falsifying the (non-)injective agreement of the Initiator. This is because the agent sends the same message regardless of whether it is the Initiator or the Receiver." A similar attack is possible on the (non-)injective agreement of the Responder.

Finally, note that the attack trace in the Figure 2.2 is the OTR3 theory with the (now commented) restriction:

restriction onekeypername:
All $A\,\#i\,\#j$. Register$(A)@i \wedge$ Register$(A)@j \implies \#i = \#j$

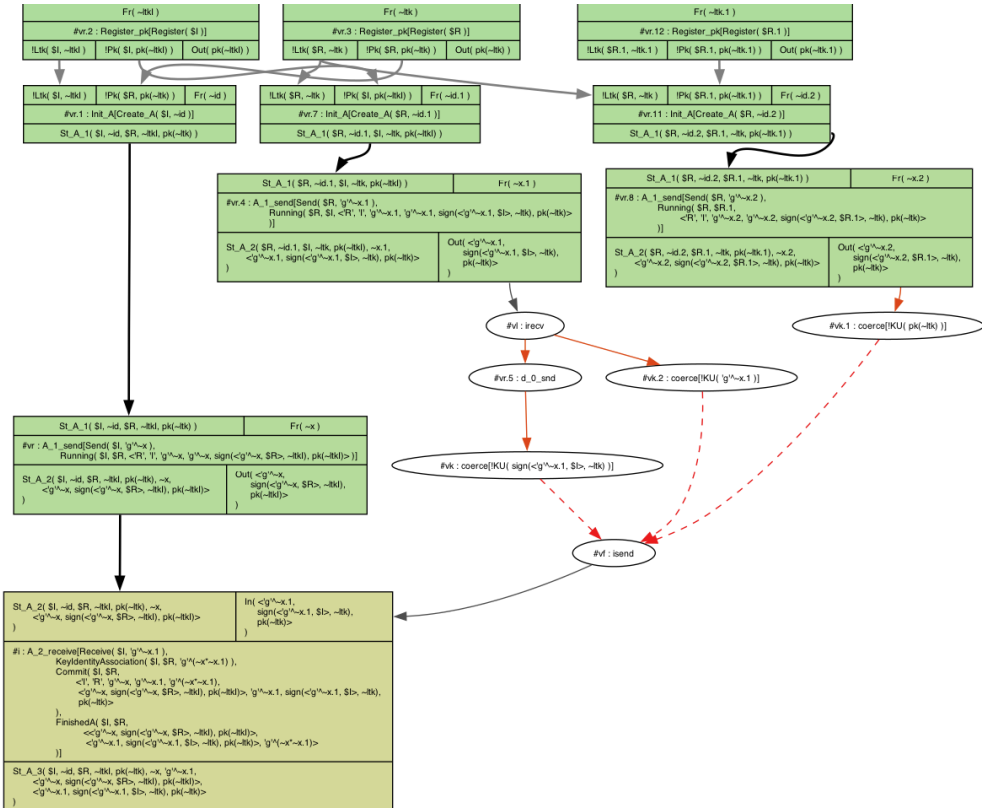


Figure 2.2: Attack against (non-)injective agreement of initiator in OTR3

Also, note that Tamarin spawns an additionnal agent to whom Y_I sends the first message with another key share to derive $\text{pk}(sk_Y)$ while it could have done so directly from the message to X_R, or from the Register_pk rule, this is due to the divided Intruder nature in Tamarin, and thus the new agent can be ignored.

2.4 SIGMA

We model our protocol OTR4 as follows:

$$
\begin{aligned}
\text{A} \rightarrow \text{B} \quad &: g^x \\
\text{B} \rightarrow \text{A} \quad &: g^y \\
\text{A} \rightarrow \text{B} \quad &: \text{A}, \text{sign}(\langle g^y, g^x \rangle, \text{sk}_A), \text{MAC}_{K_m}(\text{“0”}, \text{“A”}), pk(\text{sk}_A) \\
\text{B} \rightarrow \text{A} \quad &: \text{B}, \text{sign}(\langle g^x, g^y \rangle, \text{sk}_B), \text{MAC}_{K_m}(\text{“1”}, \text{“B”}), pk(\text{sk}_B)
\end{aligned}
$$

Discussion :

In the following, we analyze the resulting improved protocol for all standard properties.

Authentication :

For the Initiator, the (non-)injective agreement holds, because for the Initiator to commit, the Receiver must have signed the two key shares with the signing key binded to the identity he sent , and maced a message (including his identity) with a key derived from g^{xy}, preventing the previous misbinding attack, and proving he was running as the Receiver indeed.
However, (non-)injective agreement does not hold for the Receiver: An Initiator A wants to run the protocol with a Receiver B_1, the Intruder intercepts the message and sends it to another Receiver B. When A will receive the last message, it will know that it wasn't running with B_1 and therefore won't commit, while B has already commited, thinking A was running with him. More specifically, consider the following scenario: A wants to initiate a protocol run with B_1, the Intruder intercepts it and forwards it to B

$$\begin{aligned} \mathrm{A} \to B_1 &: g^x \\ \mathrm{I} \to \mathrm{B} &: g^x \end{aligned}$$

B sends its key share (g^y) to A, then A sends the next signed and maced message with the derived shared DH key to B_1, the Intruder intercepts it again and forwards it to B:

$$\begin{aligned} \mathrm{A} \to \mathrm{I} &: \mathrm{A}, \mathrm{sign}(\langle g^y, g^x \rangle, \mathrm{sk}_A), \mathrm{MAC}_{K_m}(\text{“0”}, \text{“A”}), pk(\mathrm{sk}_A) \\ \mathrm{I} \to \mathrm{B} &: \mathrm{sign}(\langle g^y, g^x \rangle, \mathrm{sk}_A), \mathrm{MAC}_{K_m}(\text{“0”}, \text{“A”}), pk(\mathrm{sk}_A) \end{aligned}$$

Since B derived the correct DH symmetric key (g^{xy}) and thinks A initiated with him, he will commit at this point even though A was never running with him, thus falsifying the (non-)injective agreement of the Receiver. Furthermore, if the Intruder forwarded B's message, A would not be tricked because the identity (sent in the MAC) would be B instead of B_1, and thus A will not commit.

Also, note that Tamarin spawns an additionnal agent to whom A sends the first message with another key share to derive pk(sk_A) while it could have done so directly from the message to B_1, or from the Register_pk rule, this is due to the divided Intruder nature in Tamarin, and thus the new agent can be ignored.

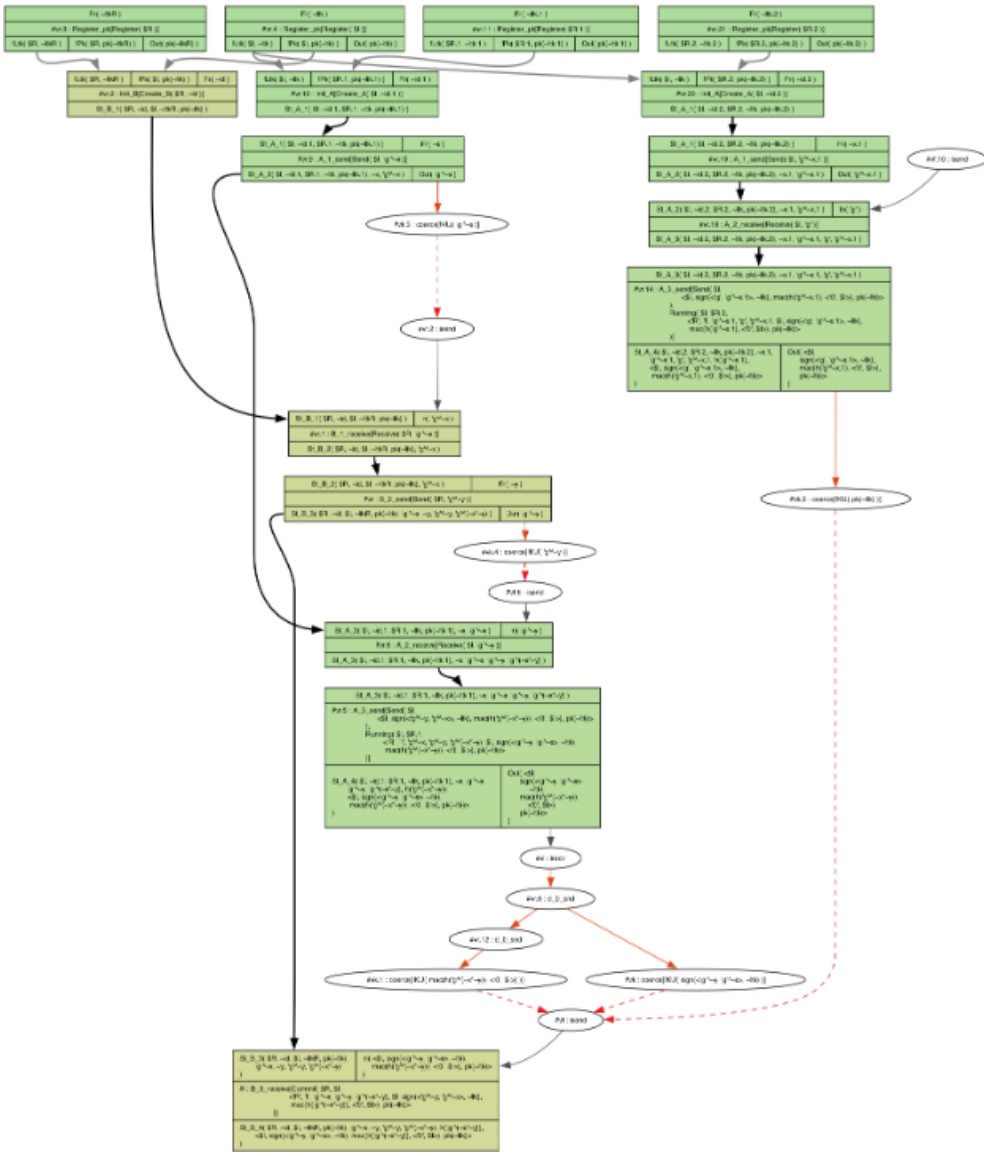


Figure 2.3: Attack against (non-)injective agreement of the Responder in the SIGMA protocol

Secrecy :

The secrecy and Perfect Forward Secrecy in OTR4 hold since we assume the agents only accept valid DH key shares, the shares are fresh at each protocol run, and the order of the DH key shares in the signatures are reversed for the Initiator and the Receiver.

What would have changed if the order of the DH keys shares in the signatures was the same for both directions ?

Well, actually, no standard property would hold!
Consider the claimed vulnerable protocol:

$$\begin{aligned}
&\mathrm{A} \to \mathrm{B} \quad : g^x \\
&\mathrm{B} \to \mathrm{A} \quad : g^y \\
&\mathrm{A} \to \mathrm{B} \quad : \mathrm{A}, \mathrm{sign}(\langle g^x, g^y \rangle \mathrm{sk}_A), \mathrm{MAC}_{K_m}(\text{“0”}, \mathrm{A}), pk(\mathrm{sk}_A) \\
&\mathrm{B} \to \mathrm{A} \quad : \mathrm{B}, \mathrm{sign}(\langle g^x, g^y \rangle, \mathrm{sk}_B), \mathrm{MAC}_{K_m}(\text{“1”}, \mathrm{B}), pk(\mathrm{sk}_B)
\end{aligned}$$

Consider the agent X, X_I (X in the role of the Initiator) wants to run with X_R (X in the role of the Receiver), the attack is as follows : X_I sends its DH key share, the Intruder intercepts it :

$$X_I \ \to \mathrm{I} : g^x$$

The Intruder sends the base g to X_I:

$$\mathrm{I} \to X_I : g$$

Then, X_I sends the signature and the mac to X_R, the Intruder intercepts the message. Moreover, **because the order of the DH key shares in the signature is the same for both directions, the Intruder can send it back and it will be accepted by** X_I . Moreover, the Intruder is able to compute the shared key K_m because it relies on the DH shared key (K_m = h(g^{xy})), which is not secret ,and thus computes the correct mac. (Note: The attacker could have produced a fresh DH pair and still succeed, Tamarin just happened to choose the key share of the Intruder to be equal to g). The Intruder can therefore craft the correct message and send it to X_I, which will claim the secrecy and commit.

$$\begin{aligned}
X_I &\to \mathrm{I} \quad : \mathrm{X}, \mathrm{sign}(g^x, g, \mathrm{sk}_X), \mathrm{MAC}_{K_m}(\text{“0”}, \mathrm{X}), pk(\mathrm{sk}_X) \\
\mathrm{I} &\to X_I \quad : \mathrm{X}, \mathrm{sign}(g^x, g, \mathrm{sk}_X), \mathrm{MAC}_{K_m}(\text{“1”}, \mathrm{X}), pk(\mathrm{sk}_X)
\end{aligned}$$

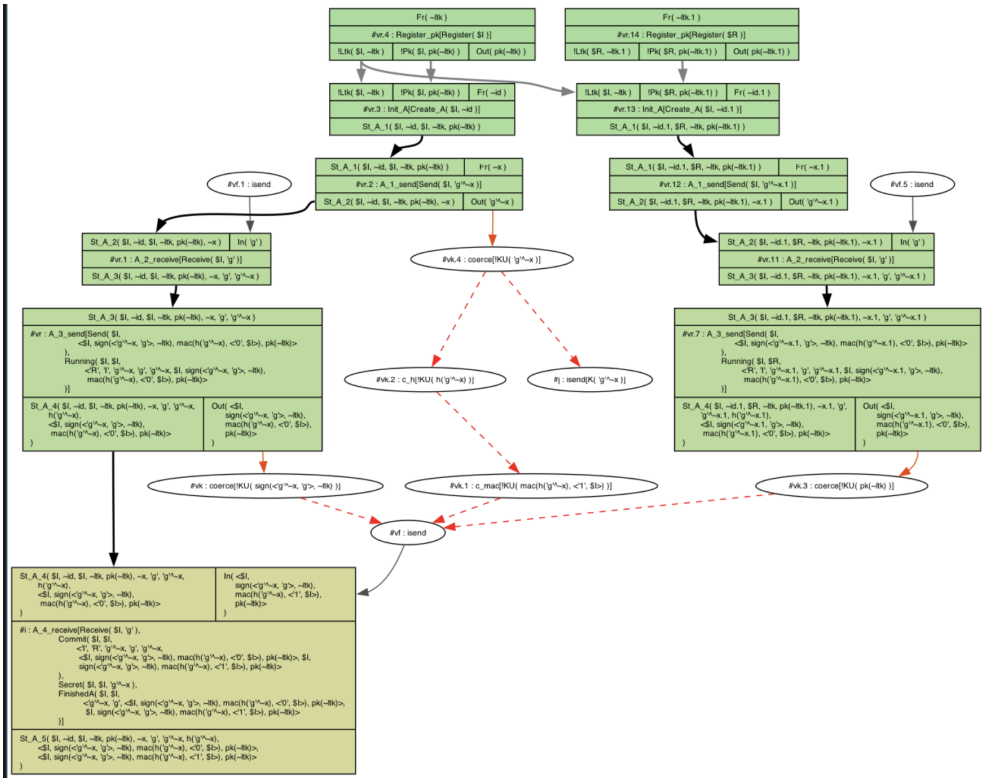


Figure 2.4: Attack when using same order of DH key shares

Also, note that Tamarin spawns an additionnal agent to whom X_I sends the first message with another key share to derive $\text{pk}(sk_X)$ while it could have done so directly from the message to X_R, or from the Register_pk rule, this is due to the divided Intruder nature in Tamarin, and thus the new agent can be ignored.